Arduino y matriz de leds + Ionic

Introducción

En este artículo vamos a comunicar, mediante Bluetooth, un móvil y una placa Arduino, a la que conectaremos una matriz de leds, para, mostrar dibujos en dicha matriz enviándolos desde el móvil.

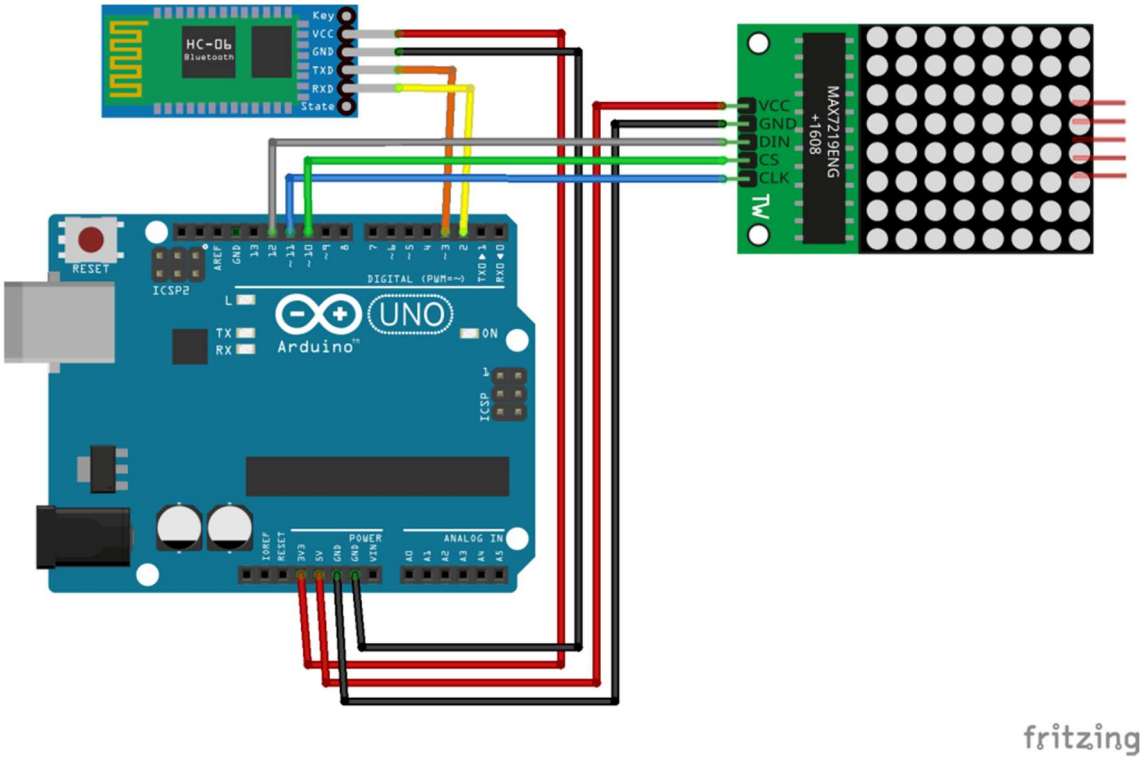
Componentes

- Arduino UNO para controlar el dispositivo Bluetooth y la matriz de leds.
- Una matriz de leds 8 x 8 y el integrado MAX2719 como controlador.
- Un módulo Bluetooth HC-06

Librerías de programación

- Ionic 5
- Angular 8

Esquema (Fritzing)



El módulo Bluetooth HC-06

El módulo **HC-06** (http://wiki.sunfounder.cc/index.php?title=Bluetooth_Transceiver_Module_HC-06) nos permite dar conectividad mediante Bluetooth a Arduino. El HC-06 es un módulo esclavo, es decir, que sólo puede ser emparejado. Para poder emparejar dispositivos (tal como haríamos con un smartphone) se requiere del módulo HC-05. Dispone de los siguientes pines:

- **VCC**: alimentación, lo conectaremos a la salida de 3.3v de Arduino.
- **GND**: masa, lo conectaremos a cualquiera de los terminales GND de Arduino
- **RXD**: pin para recibir datos, lo conectaremos al pin que definamos en Arduino para enviar datos
- **TXD**: pin para transmitir datos, lo conectaremos al pin que definamos en Arduino para recibir datos

Los pins **RXD** y **TXD** los definiremos por software mediante una librería.

La matriz de leds y el integrado MAX7219

Para el control de la matriz usaremos el integrado **MAX7219** (https://www.best-microcontroller-projects.com/max7219.html), el cual nos permite manejar hasta 64 leds usando sólo 3 pines de la placa Arduino. Las conexiones de los pins serían las siguentes:

- **VCC**: alimentación, lo conectaremos a la salida de 5v de Arduino.
- **GND**: masa, lo conectaremos a cualquiera de los terminales GND de Arduino
- **DIN**: entrada de datos, lo conectaremos a cualquier pin digital de Arduino.
- **CS**: chip select, como el MAX7219 usa **SPI (https://es.wikipedia.org/wiki/Serial_Peripheral_Interface)** para comunicarse, usa este pin para indicar a qué dispositivo de los conectados debe enviar los datos, lo conectamos a cualquier pin digital de Arduino.
- **CLK**: reloj, define el momento para el envío y recepción de datos, lo conectamos a cualquier pin digital de Arduino.

Los pins **DIN**, **CS** y **CLK** los definiremos por software mediante una librería.

La librería SoftwareSerial

Arduino UNO tiene dos pines para comunicar con dispositivos serie, el 0 (para la recepción) y el 1 (para la transmisión). Para utilizar otros pines para comunicar por puerto serie podemos usar la librería **SoftwareSerial** (https://www.arduino.cc/en/Reference/SoftwareSerial). Esta librería permite definir dos pines de Arduino como recepción y transmisión, e inicializar un puerto serie apuntando a estos pines. Con esto podremos usar más de un dispositivo serie en Arduino UNO.

Para este caso no sería necesario usarla, ya que podríamos conectar el Bluetooth a los pines 0 y 1, pero, para poder ir subiendo el código a la placa sin tener que ir conectando y desconectando USB y Bluetooth, y, dado que el código es mínimo,

queda espacio para añadir esta librería. En proyectos donde se necesite optimizar al máximo el espacio y rendimiento y, siempre que sólo se necesite conectar un dispositivo serie, podemos prescindir de ella.

La librería LedControl

Para controlar la matriz de leds mediante el integrado MAX2719 usaremos la librería **LedControl** (https://www.arduinolibraries.info/libraries/led-control), programada por **Eberhard Fahle** (https://github.com/wayoda). Esta librería nos da una serie de métodos que facilitan la programación el manejo de la matriz de leds.

Sketch de Arduino

Incluimos las librerías y declaramos los objetos necesarios para el puerto Bluetooth y el controlador de leds. También declaramos una variable para ir almacenando el texto que recibiremos a través de la conexión Bluetooth.

Para el controlador de leds, al declarar el objeto, hay que pasarle cuatro parámetros:

- El pin de Arduino donde conectamos a DIN en el controlador, en nuestro caso, el 12.
- El pin de Arduino donde conectamos a CLK en el controlador en nuestro caso, el 11,
- El pin de Arduino donde conectamos a CS en el controlador, en nuestro caso, el 10,
- El número de dispositivos (matrices) que vamos a conectar, en este caso, 1.

Para crear el puerto serie para comunicar con Bluetooth se han de pasar dos parámetros:

- El pin por donde se transmitirán los datos, en nuestro caso, el 3.
- El pin por donde se recibirán los datos, en nuestro caso, el 2.

```
#include <SoftwareSerial.h>
#include <LedControl.h>

LedControl lc = LedControl(12, 11, 10, 1); // DIN, CLK, CS, Device
SoftwareSerial BT(3, 2); // 3 RX, 2 TX
String pictureData = "";
```

En el setup del sketch vamos a iniciar el puerto serie para el Bluetooth, pasándole 9600 como parámetro para los bits por segundo, y, para el controlador de leds, vamos a establecer la intensidad de los leds, limpiaremos la pantalla y activaremos el ahorro de energía. Todos los métodos que usamos del controlador de leds tienen un primer parámetro que se usa para indicar a qué dispositivo de los conectados al bus SPI queremos aplicar la acción. Como toda lista en programación, empieza por el 0, por lo que el primer dispositivo (y en nuestro caso, el único), se identifica con ese 0.

```
void setup() {
          BT.begin(9600);
          lc.setIntensity(0, 2);
          lc.clearDisplay(0);
          lc.shutdown(0, true);
}
```

En el loop del sketch vamos a comprobar si el puerto serie Bluetooth está disponible para su lectura, si es así, leeremos el valor que hayamos recibido y...

- ...si es un final de línea, ejecutaremos la función **action()**, que ejecutará lo que convenga según los datos recibidos.
- ...si es otro valor, lo almacenará en la variable global declarada para almacenar el texto recibido.

```
void loop() {
          if (BT.available()) {
                    char character = BT.read();
                    if (character != '\n') {
                              pictureData.concat(character);
                    } else {
                              action();
                    }
          }
}
```

La función **action()** comprueba el contenido de la variable encargada de almacenar los datos recibidos del puerto serie Bluetooth y, en caso de recibir la cadena clean, llamará a la función **clean()** para limpiar la matriz, si no, llamará a la función **draw()** para pintar la matriz.

```
void action() {
          if (pictureData == "clean") {
                    clean();
          } else {
                    draw();
          }
}
```

La función **clean()** simplemente limpia la matriz, activa el ahorro de energía y asigna una cadena vacía a la variable que almacena los datos.

```
void clean() {
          lc.clearDisplay(0);
          lc.shutdown(0, true);
          pictureData = "";
}
```

La función **draw()** es la encargada de pintar en la matriz la imagen recibida a modo de datos por el puerto serie Bluetooth. Primero desactiva el modo de energía y limpia la pantalla, y, seguidamente, recorre toda la cadena de texto recibida y, por cada carácter de la cadena, calcula la posición en la matriz que le corresponde y enciende o apaga el led de dicha posición según sea el carácter un 1 o un 0.

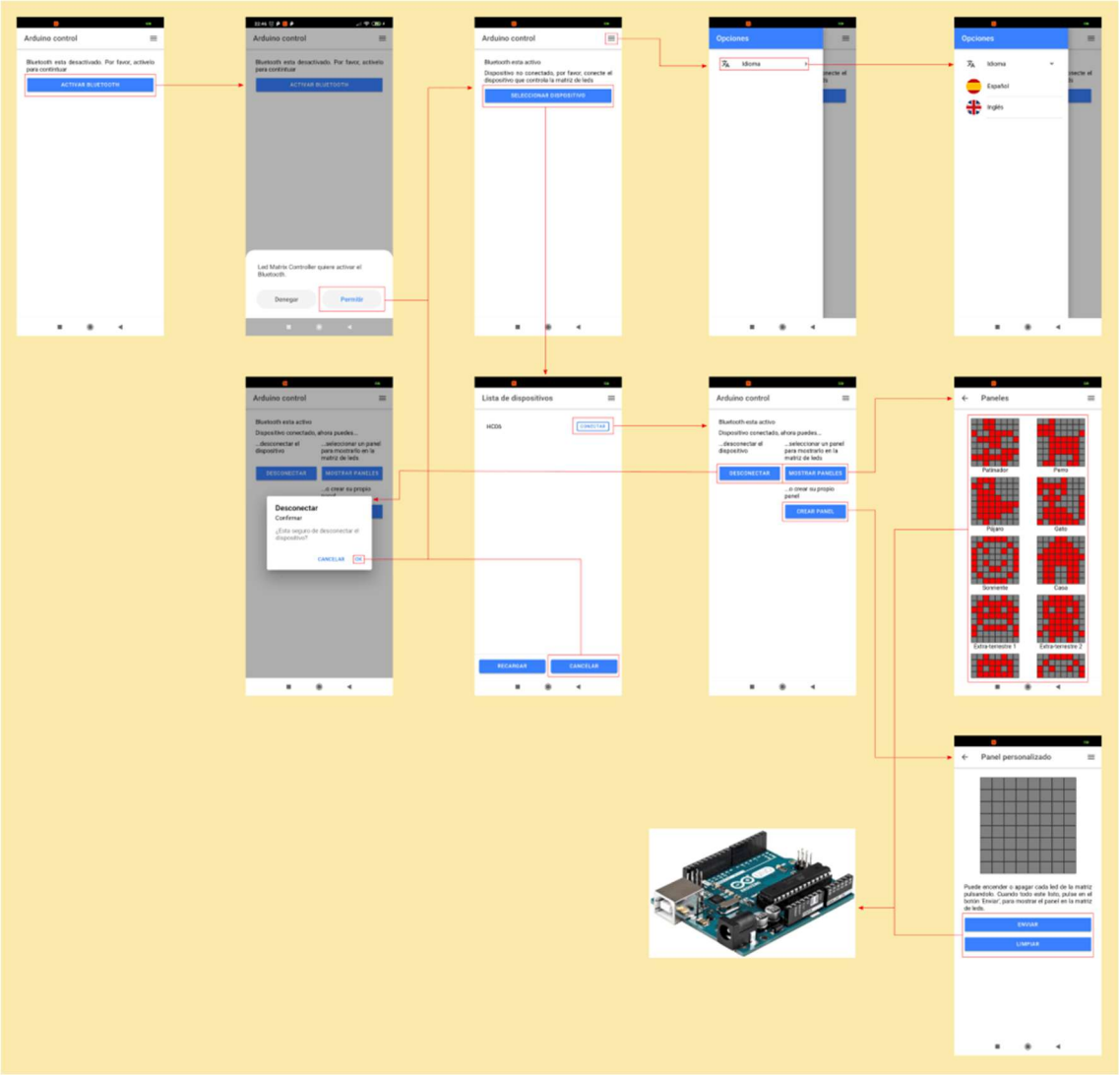
Finalmente, vacía la variable que almacena los datos recibidos.

```
void draw() {
          lc.shutdown(0, false);
          lc.clearDisplay(0);
          int total = pictureData.length();
          int byLine = (int)sqrt(total);
          for(int i = 0; i < total; i++) {
                    boolean data = pictureData.charAt(i) == '1' ? true : false;
                    int row = i / byLine;
                    int col = i % byLine;
                    lc.setLed(0, row, col, data);
          }
          pictureData = "";
}
```

Aplicación para el móvil

Para controlar qué imagen se muestra en la matriz usaremos una aplicación para móvil programada usando Ionic 5 y Angular 8.

Mapa de la aplicación



Inicio

Al iniciarse la aplicación se establece el idioma de la aplicación, adaptándose a la configuración del dispositivo, en caso de que exista en la lista de lenguajes disponibles. En caso que no estuviera disponible se usará el lenguaje por defecto.

También se crea una suscripción al evento de reanudación de la plataforma para comprobar el estado de la conexión Bluetooth. Esto es útil por si el usuario sale de la aplicación, realiza operaciones con la configuración del Bluetooth y vuelve a la aplicación.

```
if (this.availableLanguages.includes(this.translate.getBrowserLang())) {
          this.translate.setDefaultLang(this.translate.getBrowserLang());
} else {
          this.translate.setDefaultLang(this.defaultLanguage);
}
this.platform.resume.subscribe(() => {
          // the user returns to the application,
          // must be checked the status of the bluetooth device
          this.logger.messageConsole(`Application resumed`, this.constructor.name);
          this.bt.stateBT();
          this.bt.connectedBT();
});
```

Home (sin dispositivo conectado)

Al iniciarse la página **home** la aplicación se suscribe al estado de dos propiedades del servicio que gestiona estados y operaciones con Bluetooth para comprobar su estado. Esto permite saber si esta habilitado y si está conectado con algún dispositivo externo. En esta página, esta comprobación, se realiza para modificar su aspecto mostrando u ocultando según qué botones acorde con el estado de la conexión, pero, en otras páginas también se usará para devolver al usuario a la página **home** en caso de que el dispositivo no esté habilitado o conectado.

```
ngOnInit(): void {
          this.stateSubscription = this.bt.state$.subscribe((res) => {
                    this.logger.messageConsole(`stateBT: ${res}`, this.constructor.name);
                    this.zone.run(() => {
                              this.state = res;
                    });
          });
          this.connectedSubscription = this.bt.connected$.subscribe((res) => {
                    this.logger.messageConsole(`connectedBT: ${res}`,
this.constructor.name);
                    this.zone.run(() => {
                              this.connected = res;
                    });
          });
}
```

El estado de estos valores se consulta cuando la aplicación ha finalizado el proceso de entrar en la página:

```
ionViewDidEnter(): void {
          this.bt.stateBT();
          this.bt.connectedBT();
}
```

Listar dispositivos

Al pulsar sobre el botón **Seleccionar dispositivo** la aplicación navega hacia la página para listar dispositivos, donde, además de las suscripciones para comprobar el estado de la conexión Bluetooth, se añade una más para comprobar la lista de dispositivos Bluetooth encontrados.

```
this.unpairedSubscription = this.bt.unpaired$.subscribe(
          (res) => {
                    this.zone.run(() => this.unpairedList = res);
          },
          (error) => {
                    this.logger.messageConsole(`Error: ${error}`, this.constructor.name);
          },
);
```

La aplicación busca dispositivos Bluetooth al entrar en la página o si el usuario pulsa el botón **Recargar**.

```
ionViewDidEnter(): void {
          this.discoverUnpaired();
}
```

```
<ion-button expand="block" (click)="discoverUnpaired()">
          {{ 'common.reload' | translate }}
</ion-button>
```

```
discoverUnpaired(): void {
          this.unpairedList = [];
          this.bt.discoverUnpaired();
}
```

Home (con dispositivo conectado)

Al conectarse a un dispositivo Bluetooth, la página home cambia su aspecto mostrando los botones para navegar a las páginas que permiten enviar imágenes a mostrar a la matriz y el botón para desconectar el dispositivo Bluetooth.

Al pulsar sobre el botón **Desconectar** llamamos a un método que, usando el componente **AlertController**, nos permite mostrar un diálogo de confirmación, que llamará la método **disconnectDevice** del servicio para el manejo de Bluetooth, si el usuario confirma la acción.

```
async disconnectDevice(): Promise<any> {
          const alert = await this.alertCtrl.create({
                    header: this.translate.instant('common.disconnect'),
                    subHeader: this.translate.instant('common.confirm'),
                    message: this.translate.instant('home.disconnect'),
                    buttons: [
                              {
                                        text: this.translate.instant('common.cancel'),
                                        role: "cancel",
                              },
                              {
                                        text: this.translate.instant('common.ok'),
                                        handler: () => {
                                                  this.bt.disconnectDevice();
                                        },
                              },
                    ],
          });
          await alert.present();
}
```

Páginas para enviar imágenes a Arduino

La aplicación tiene dos páginas para enviar imágenes a la matriz:

- **panel**: carga un fichero JSON con información de imágenes para enviar a la matriz.
- **custom-panel**: permite definir un dibujo y enviarlo a la matriz.

Panel

Panel carga el contenido del fichero JSON usando el plugin **File** (https://ionicframework.com/docs/native/file).

```
async ionViewDidEnter(): Promise<any> {
          await this.loading.showLoading(
                    this.translate.instant("panels.loadingPanels")
          );
          this.file
          .readAsText(
                    `${this.file.applicationDirectory}www/assets/`,
                    "panels-data.json"
          )
          .then((data: string) => {
                    this.panels = JSON.parse(data);
          })
          .catch((error) => {
                    this.logger.messageConsole(`Error: ${error}`, this.constructor.name);
                    this.toast.showShortBottom(
                              this.translate.instant("panels.errorLoadingPanels")
                    );
          })
          .finally(() => {
                    setTimeout(async () => {
                              await this.loading.hideLoading();
                    }, 1000);
          });
}
```

Panel personalizado

Custom-panel usa una matriz donde almacena el estado de los leds para, posteriormente, enviarlo a Arduino.

```
initPanel(): Array<Array<number>> {
          return [
                    [0, 0, 0, 0, 0, 0, 0, 0],
                    [0, 0, 0, 0, 0, 0, 0, 0],
                    [0, 0, 0, 0, 0, 0, 0, 0],
                    [0, 0, 0, 0, 0, 0, 0, 0],
                    [0, 0, 0, 0, 0, 0, 0, 0],
                    [0, 0, 0, 0, 0, 0, 0, 0],
                    [0, 0, 0, 0, 0, 0, 0, 0],
                    [0, 0, 0, 0, 0, 0, 0, 0],
          ];
}
```

El componente app-grid-panel

Las dos páginas usan el componente **app-grid-panel**, pero, adaptándolo a sus necesidades mediante parámetros.

En la página **panel** se usa de la siguiente manera:

```
<app-grid-panel
[leds]="item.data"
[name]="getPanelName(item)"
(click)="showPanel(item)"
>
```

Pasándole los datos de estado de los leds y el nombre del panel. También se define la acción para el evento **click**, en este caso será enviar la información del panel a Arduino.

En la página **custom-panel** se usa de la siguiente manera:

```
<app-grid-panel
[leds]="panel"
[custom]="true"
(changeLed)="updatePanel($event)">
</app-grid-panel>
```

Pasándole los datos de estado de los leds y custom, para indicar que es un panel personalizado. El evento **changeLed** se asocia a un método de la página que actualiza la matriz de estados de leds, este cambio se verá reflejado en el componente **app-grip-panel**.

En la página **custom-panel**, el envío de datos hacia Arduino se hace mediante botones, uno para enviar la imagen, el cual envía la información de la matriz, y otro, para limpiar la matriz, que envía el comando **clean**.

El servicio BtService

Toda la gestión y manejo de operaciones relacionadas con Bluetooth se realiza mediante el servicio BtService. Este dispone de los siguientes métodos:

- **enableBT**: habilita la conexión Bluetooth del móvil.
- **stateBT**: comprueba el estado de la conexión Bluetooth.
- **connectedBT**: comprueba si el móvil está conectado a algún dispositivo externo.
- **discoverUnpaired**: busca dispositivos externos para conectarse. Se añaden a un lista que puede consultarse posteriormente.
- **connectDevice**: conecta uno de los dispositivos de la lista obtenida a partir de **discoverUnpaired** al móvil.
- **disconnectDevice**: desconecta el dispositivo enlazado anteriormente usando **connectDevice**.
- **sendData**: envía datos a través de Bluetooth al dispositivo externo.

También dispone de tres propiedades observables:

- **state$**: devuelve true si la conexión está habilitada y false en caso contrario.
- **connected$**: devuelve true si el móvil está conectado a un dispositivo externo y false en caso contrario.
- **unpaired$**: devuelve la lista de dispositivos disponibles para conectar.